PROBLÈME · ANNALES SPÉ · 3 HEURES

Sous-graphe le plus dense

1. Réseaux, chemins augmentants et plus petite bordure

Langage C (ou pseudo-code de même niveau). Un réseau $H = (W, F, \mu, s, t)$: sommets numérotés de 0 à $n+1$, source $s = 0$, puits $t = n+1$, multiplicité $\mu(e) \in \mathbb{N}^*$ pour chaque arc $e \in F$. Fichier `network.h` :

```
#ifndef NETWORK_H
#define NETWORK_H
const unsigned int NMAX = 102;
struct network_s { unsigned int n; unsigned mu[NMAX][NMAX]; };
typedef struct network_s network;
void nw_init(unsigned int n);
void nw_add(int u, int v);
void nw_remove(int u, int v);
void nw_disconnect(void);
void _arr_init(int *T);
bool _bfs_tree(void);
void _residue(void);
#endif
```

Fichier `network.c` : `network H; int A[NMAX];` ; `H.mu[u][v]` vaut $\mu((u,v))$, ou 0 si (u,v) n'est pas un arc.

1. Rôle des lignes `#ifndef`, `#define` et `#endif`.
2. Comment faire interagir `network.h` et `network.c` ?
3. `void nw_init(unsigned int n)` : précondition $n < n_{\max}$ vérifiée par une assertion ; `H` devient le réseau vide à $n+2$ sommets.
4. `void nw_add(int u, int v)` : incrémente $\mu[u][v]$.
5. `void nw_remove(int u, int v)` : retire une occurrence, avec une assertion si l'arc n'existe pas.
6. `void _arr_init(int *T)` : met les $n+2$ cases de `T` à -1.
7. `bool _bfs_tree(void)` : calcule dans `A` une arborescence de plus courts chemins (parcours en largeur depuis s) : `A[s] = s`, `A[v]` = prédécesseur pour v accessible, -1 sinon ; renvoie `true` si t est accessible.

Un ensemble Π de k chemins de s à t est réalisable si chaque arc e est utilisé au plus $\mu(e)$ fois. Le réseau résiduel $\mathrm{Res}(H, \Pi)$ s'obtient en retirant, pour chaque arc (u,v) de chaque chemin, une occurrence de (u,v) et en ajoutant une occurrence de (v,u).

1. `void _residue(void)` : remplace `H` par $\mathrm{Res}(H, \{\alpha\})$, où α est le chemin de s à t dans l'arborescence `A`.
2. Après k appels successifs (chemins $\alpha_1, \dots, \alpha_k$), montrer qu'il existe un ensemble réalisable Π de k chemins tel que le réseau obtenu soit $\mathrm{Res}(H, \Pi)$.
3. `void nw_disconnect(void)` : tant qu'il existe un chemin de s à t, remplacer H par le résiduel le long d'un plus court chemin ; à la fin, `A` est une arborescence de racine s n'atteignant pas t.
4. Montrer que `nw_disconnect` termine (variant inspiré de 9).

Bordure ∂S : arcs $(u, v) \in F$ avec $u \in S$, $v \notin S$; $\mu(\partial S) = \sum_{e \in \partial S} \mu(e)$. Un entourage est un $S \subseteq W$ avec $s \in S$, $t \notin S$.

1. Pour Π réalisable de k chemins et S entourage : $k \leqslant \mu(\partial S)$.

On fixe Π_0 réalisable de k_0 chemins tel que $\mathrm{Res}(H, \Pi_0)$ n'ait plus de chemin de s à t, et $S_0 =$ sommets accessibles depuis s dans $\mathrm{Res}(H, \Pi_0)$.

1. Pour tout arc (u, v) avec $u \in S_0$, $v \notin S_0$: toutes ses $\mu(e)$ occurrences sont utilisées par Π_0.
2. Aucune occurrence d'un arc (v, u) avec $v \notin S_0$, $u \in S_0$ n'est utilisée par Π_0.
3. Montrer $k_0 = \mu(\partial S_0)$.
4. Montrer $k_0 = \sigma(H)$ (plus petite bordure d'un entourage) et que, après `nw_disconnect`, $\{v : A[v] \geqslant 0\}$ est un entourage de plus petite bordure.

2. Sous-graphe le plus dense : l'algorithme de Goldberg

Graphe non orienté $G = (V, E)$ à sommets $1, \dots, n$, listes d'adjacence doublement chaînées : `struct link_s { int node; struct link_s *prev, *next; }` ; `struct graph_s { unsigned int n; link **neighbours; }`, tableau de longueur $n + 1$ (case 0 inutilisée) ; chaque arête est stockée comme deux arcs. Densité $\delta(G) = \frac{m}{n}$; $\rho(G) = \max_{X \neq \emptyset} \delta(G_X)$, où G_X est le sous-graphe induit par X.

1. `graph *gr_create(unsigned int n)` : graphe vide à n sommets.
2. `void gr_add(graph *G, int u, int v)` : insère l'arête $\{u, v\}$.
3. `int degree(graph *G, int v)`.
4. `int edges(graph *G)` : nombre d'arêtes.
5. `double density(graph *G)`.

Réseau associé à G (n sommets, m arêtes) et à un entier r : $W = V \cup \{s, t\}$; arc (s, v) de multiplicité mn^2 pour tout v ; arcs (u, v) et (v, u) de multiplicité n^2 pour toute arête ; arc (u, t) de multiplicité $mn^2 - n^2 \deg_G(u) + 2r$ pour tout u. Question : a-t-on $\rho(G) > \frac{r}{n^2}$?

1. `void reduce(graph *G, int r)` : place dans `H` le réseau associé.
2. Montrer $\delta(G_X) = \dfrac{\sum_{v\in X} \deg_G(v) - \mathrm{card}\{\{u,v\} \in E : u \in X,\ v \notin X\}}{2|X|}$.
3. Pour $S = \{s\} \cup X$: $\mu(\partial S) = mn^3 + 2|X|(r - n^2\delta(G_X))$, et $\mu(\partial\{s\}) = mn^3$.
4. Montrer : $\sigma(H) = mn^3 \iff \rho(G) \leqslant \frac{r}{n^2}$ et $\sigma(H) < mn^3 \iff \rho(G) > \frac{r}{n^2}$.
5. `bool oracle(graph *G, int r)` : renvoie vrai si et seulement si $\rho(G) > \frac{r}{n^2}$. Justifier.

$\Delta = \{(m', n') : 0 \leqslant m' \leqslant m,\ 1 \leqslant n' \leqslant n\}$.

1. Si $\frac{m_1}{n_1} \neq \frac{m_2}{n_2}$ (couples de Δ), alors $\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| \geqslant \frac{1}{n^2}$.
2. S'il n'existe aucun sous-graphe induit de densité $\geqslant \delta(G_X) + \frac{1}{n^2}$, alors $\delta(G_X) = \rho(G)$.
3. `void binary_search(graph *G)` : modifie `A` pour que $X = \{v \in V : A[v] \geqslant 0\}$ vérifie $\delta(G_X) = \rho(G)$. Justifier.
4. Comment faire interagir `densest_subgraph.c` avec `network.h` et `network.c` ?

3. Algorithme glouton de Charikar

Algorithme de Charikar : $\Gamma \leftarrow G$, $G_n \leftarrow G$; pour i de n à 2 : si $\delta(G_i) > \delta(\Gamma)$ alors $\Gamma \leftarrow G_i$; on retire de G_i un sommet v_i de degré minimal (et ses arêtes) pour former G_{i-1} ; on renvoie Γ.

Une orientation de G choisit un sens pour chaque arête ; $\deg^-(v)$ est le degré entrant. $\widehat{G_0}$ oriente l'arête $\{v_i, v_{i'}\}$ en $(v_{\min(i,i')}, v_{\max(i,i')})$, c'est-à-dire vers le sommet dont le retrait l'a supprimée.

1. Montrer $\delta(G) \leqslant \max_v \deg^-_{\widehat{G}}(v)$ pour toute orientation $\widehat{G}$ de G.
2. Montrer $\deg^-_{\widehat{G_0}}(v_i) \leqslant 2\delta(G_i)$ et en déduire un facteur d'approximation de l'algorithme.
3. Détailler une structure de données qui exécute l'algorithme en $O(n+m)$ (suggestion : une table des sommets de degré d pour chaque $d \in [\![0, n]\!]$).